ESC401 High Performance Computing

Course Summary

UZH - Dario Hug

January 13, 2026

Contents

1 Introduction to High Performance Computing

1.1 Overview

High Performance Computing (HPC) deals with solving the *same computational problem* faster or at larger scale by using multiple computing resources simultaneously. These resources can range from multi-core CPUs on a single machine to large supercomputers with thousands of interconnected nodes. HPC is widely used in scientific simulations, data analysis, and engineering applications.

1.2 Ways to Achieve High Performance

Several approaches are used to speed up computations:

- **Multiple threads**: Parallel execution within a single process, usually sharing memory.
- **Multiple processes**: Independent processes that may communicate via message passing.
- **Message Passing (MPI)**: Explicit communication between processes, locally or across nodes.
- **GPU Acceleration**: Using GPUs as accelerators for massively parallel workloads.
- **Batch clusters and cloud services**: Computing resources accessed via job schedulers or cloud platforms (e.g. ScienceCloud).

Dedicated or domain-specific HPC codes are common, such as `Genga` or `RAMSES`, while others use accelerators in hybrid setups (e.g. `pkdgrav3`, GRAPE, Intel Xeon Phi).

1.3 Supercomputers

A supercomputer typically consists of:

- Many compute nodes connected together
- A fast, specialized interconnect (network)
- A Unix/Linux operating system
- Non-interactive usage via batch job systems

Supercomputers are usually accessed remotely and are physically far away from the user.

1.4 Remote Access and SSH

Remote access to HPC systems is commonly done using the **SSH (Secure Shell)** protocol, which provides encrypted communication. Key concepts:

- Servers have public host keys to prevent man-in-the-middle attacks
- Host keys are stored in `~/.ssh/known_hosts`
- Public/private key pairs are used for authentication

Important SSH-related files and tools:

- `~/.ssh/id_rsa`: private key
- `~/.ssh/id_rsa.pub`: public key
- `authorized_keys`: keys allowed to access an account
- `ssh-keygen`: generate key pairs
- `ssh-copy-id`: install public keys on a server
- `ssh-agent`, `ssh-add`: manage loaded keys

1.5 Threads vs. Processes

- **Threads**: Share memory and address space, but have separate registers
- **Processes**: Separate memory spaces, communication requires IPC or MPI

This distinction is fundamental for understanding parallel programming models.

2 Version Control, Compilation, and Batch Systems

2.1 Git and Branching Models

Git is a distributed version control system where every clone contains the full history of the repository. Work is typically organized using branches.

A common branching strategy is **git-flow**, which is both a workflow philosophy and a set of supporting tools. It introduces well-defined branch roles:

- **master (main)**: Always contains stable, production-ready code.
- **develop**: Integration branch for completed features; reflects the next release.
- **feature branches**: Used for developing new features; branched off from `develop`.
- **release branches**: Used to prepare a release; eventually merged into `master` and `develop`.

Git repositories can have remotes hosted on platforms such as GitHub, Bitbucket, GitLab (e.g. `gitlab.uzh.ch`), or even another local directory or computer.

2.2 Basic Git Concepts

- **Working files**: Files currently checked out and possibly modified.
- **Local repository**: A complete copy of the project history, including all branches.
- **Remote repository**: A shared repository used for collaboration.

This distributed nature allows offline work and flexible collaboration.

2.3 Compilers and Machine Code

A compiler translates high-level source code (e.g. C/C++) into machine code that the CPU can execute. For example, a simple `Hello World` program written in C is eventually translated into assembly instructions and binary machine code.

Machine code is often represented in:

- **Binary**: Base-2 representation used internally by the CPU
- **Hexadecimal**: Compact base-16 representation commonly used for readability

Assembly examples often use **x86**, which refers to a widely used family of CPU architectures originally developed by Intel. x86 defines the instruction set, registers, and execution model of the processor.

2.4 Anatomy of a Compiler

A compiler is typically composed of several stages:

- **Frontend**: Lexical analysis, parsing, and semantic analysis of source code
- **Intermediate Representation (IR)**: Architecture-independent code representation
- **Optimizer**: Improves performance by transforming the IR
- **Backend**: Generates architecture-specific machine code

2.5 Compilation Process

A typical compilation workflow includes:

- Preprocessing (handling includes and macros)
- Compilation to assembly or object code
- Linking object files and libraries into an executable

In HPC, compiler choice and optimization flags have a significant impact on performance.

2.6 Modules on Eiger

Eiger uses a **module system** to manage software environments. Modules allow users to dynamically load and unload compilers, libraries, and tools.

A *loaded module* modifies the user's environment (e.g. `PATH`, `LD_LIBRARY_PATH`) so that the selected software becomes active. This avoids conflicts between different compiler and library versions.

2.7 Compiler Suites on Eiger

Eiger provides several compiler environments, each optimized for different use cases:

- **Cray Compiler (PrgEnv-cray)**: Highly optimized for Cray systems
- **GNU Compiler (PrgEnv-gnu)**: Open-source, portable, widely used
- **Intel Compiler (PrgEnv-intel)**: Optimized for Intel architectures
- **Portland Group / NVIDIA (PrgEnv-pgi)**: Important for OpenACC and GPU programming
- **Clang**: Modern compiler, GNU-compatible, used by Apple and Cray
- **Visual Studio**: Windows-focused, not typically used in HPC

Different compilers can produce significantly different performance for the same source code.

2.8 Batch Queue Systems and SLURM

HPC systems use batch queue systems to manage shared resources. **SLURM** (Simple Linux Utility for Resource Management) is the scheduler used on Eiger.

Users submit jobs rather than running programs interactively. SLURM allocates nodes, CPUs, and time according to availability and policies.

2.9 SLURM Job Management

Common SLURM commands include:

- `sbatch`: Submit a batch job script
- `squeue`: View job queue and job states
- `sinfo`: Display partition and node information
- `srun`: Launch job steps (used inside job scripts)
- `scancel`: Cancel a running or queued job

Partitions such as `normal` and `debug` define limits on runtime and resources. Resource usage is accounted for (e.g. charged per node-hour).

2.10 Processes, Threads, and Hardware Mapping

- **Node**: A physical machine within a cluster
- **CPU**: A processor on a node, consisting of multiple cores
- **Process**: Independent execution unit with its own memory space
- **Thread**: Lightweight execution unit within a process, sharing memory

Typically, MPI uses multiple processes (often one per CPU core), while threading models (e.g. OpenMP) use multiple threads within a process. Hybrid approaches combine both.

2.11 Parallel Scaling

- **Perfect scaling**: Runtime decreases proportionally with the number of processors
- **Typical scaling**: Diminishing returns due to communication overhead, synchronization, and serial code sections

Understanding scaling behavior is crucial for efficient use of HPC resources.

3 Parallel Programming Concepts and Compiler Optimization

3.1 Parallelization Goals

The main goal of parallel programming is to **accelerate computation** by distributing work across multiple threads, cores, or nodes. This can involve:

- Modifying existing code for parallel execution
- Writing new parallel algorithms from scratch

Introductory exercise: computing π in parallel using simple numerical integration.

3.2 Compiler Optimization Levels

Compilers can optimize code for speed, size, or other characteristics. For example, GCC optimization flags:

- `-O0`: No optimization, straightforward translation of source code
- `-O1, -O2, -O3`: Increasingly aggressive optimizations
- `-ffast-math`: Reorders floating-point operations to improve performance, possibly sacrificing strict IEEE precision
- `-mavx2`: Uses AVX2 vector instructions for SIMD parallelism

3.3 Floating-Point Considerations

Floating-point arithmetic is not strictly associative:

$$a + b + c \neq a + (b + c)$$

Reordering operations (as done in optimizations or vectorization) can slightly change results. The same applies to multiplication with distributive reordering:

$$ab + c \neq ab + ac$$

3.4 Separate Compilation of Modules

C programs can be split into multiple files (modules) for maintainability and reuse. Example:

```
cpi.c -> cpi.o
gettime.c -> gettime.o
Link -> cpi
Run -> ./cpi
```

$$\text{cpi.c} \xrightarrow{\text{compile}} \text{cpi.o} \quad \text{gettime.c} \xrightarrow{\text{compile}} \text{gettime.o} \quad \text{cpi.o + gettime.o} \xrightarrow{\text{link}} \text{cpi executable}$$

A `Makefile` automates this process, specifying compilation flags (`CFLAGS`) and linking flags (`LDFLAGS`):

```
CFLAGS=-Wall -O3 -ffast-math -mavx2 -fopenmp
LDFLAGS=-fopenmp

cpi: cpi.o gettime.o
    gcc $(CFLAGS) -o cpi cpi.o gettime.o $(LDFLAGS)
```

3.5 Message Passing with MPI

MPI (Message Passing Interface) enables parallelism across multiple nodes or processes. Each process in an MPI program is identified by a unique **rank**, an integer ranging from 0 to $N-1$, where N is the total number of processes. The rank determines the role of a process (e.g., sending or receiving data, performing a portion of a computation).

- **Ranks**: Unique identifiers for each process. Rank 0 is often used as the "master" process.

- **Process management**: MPI spawns and coordinates multiple processes.
- **Communication**: Processes exchange data via point-to-point or collective operations.
- **Synchronization**: Ensures that dependent computations happen in the correct order.

Core MPI functions:

- `MPI_Init`: Initialize the MPI environment
- `MPI_Comm_rank`: Returns the rank of the current process
- `MPI_Comm_size`: Returns the total number of processes
- `MPI_Bcast`: Broadcast a value from one process to all others
- `MPI_Reduce / MPI_Allreduce`: Combine values from all processes (e.g., sum) and store result in one or all ranks

Workflow Example: Computing a Parallel Sum (e.g., for π)

1. **Split work**: Each MPI rank computes a portion of the sum
2. **Compute locally**: Each rank performs its partial calculation independently
3. **Reduce**: Partial results are sent back to rank 0 (or all ranks) using `MPI_Reduce` or `MPI_Allreduce`
4. **Combine results**: Rank 0 (or all ranks) computes the final sum

Simple Visualization

$$\underbrace{\text{Data Block}}_{\text{Full Problem}} \xrightarrow{\text{Split by rank}} \begin{bmatrix} \text{Rank 0: chunk 0} \\ \text{Rank 1: chunk 1} \\ \vdots \\ \text{Rank N-1: chunk N-1} \end{bmatrix} \xrightarrow{\text{Compute}} \begin{bmatrix} \text{Partial sum 0} \\ \text{Partial sum 1} \\ \vdots \\ \text{Partial sum N-1} \end{bmatrix} \xrightarrow{\text{Reduce}} \text{Final sum (all or r0)}$$

3.6 Shared-Memory Parallelism with OpenMP

OpenMP provides thread-level parallelism within a single process:

- `#pragma omp parallel for` parallelizes loops across threads
- Requires linking and compiling with `-fopenmp` flags
- Use `OMP_NUM_THREADS` to control thread count

Example: Parallel sum for computing π requires `reduction` to avoid race conditions:

```
#pragma omp parallel for reduction(+:sum)
for (i=0; i < steps; i++) {
    double x = (i+0.5)*step;
    sum += 4.0 / (1.0+x*x);
}
```

OpenMP vs. MPI: OpenMP provides *shared-memory, thread-level parallelism* within a single node, while MPI provides *distributed-memory, process-level parallelism* across multiple nodes or computers.

3.7 Parallel Pitfalls

Critical sections: Updating shared data must be protected to prevent race conditions.

Deadlock: Occurs when processes wait for resources in a circular manner. Four necessary conditions:

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

Starvation: A thread or process may never acquire the resources it needs, often mitigated by resource hierarchies.

3.8 Load Balancing

Ensuring all threads or processes have roughly equal work prevents idle resources and improves parallel efficiency. Unequal workloads reduce scaling and can negate parallel speedup.

3.9 Summary of Parallel Scaling Concepts

- MPI: Distributed memory, process-level parallelism across nodes
- OpenMP: Shared-memory, thread-level parallelism within a node
- Hybrid: Combine MPI across nodes and OpenMP within nodes for maximal performance

4 The Shell, C Basics, and Memory Management

4.1 The Shell Prompt

A typical shell prompt looks like:

`[eiger][dpotter@nid001080 ~]$`

It consists of:

- **Hostname / cluster**: `eiger`
- **Username**: `dpotter`
- **Current directory**: `~`
- **Prompt symbol**: `$` for regular user

This prompt is where you type commands to interact with the system.

4.2 Scripting and Automation

BASH scripting helps:

- Automate repetitive tasks
- Reduce human errors
- Orchestrate program execution

Other scripting tools exist (Python, Perl), but BASH is ubiquitous in HPC. Regular expressions (RegEx) are often used for pattern matching in scripts.

4.3 C Data Types

Primitive types and sizes:

Type	Size (bits/bytes)	Description / Range
char	8 (1)	[-127,+127] or [0,255]
short	16 (2)	[-32767,+32767] or [0,65535]
int	16–32 (2–4)	Machine natural size
long	32–64 (4–8)	[-2,147,483,647,+2,147,483,647]
long long	64 (8)	[-263 1, +263 1]
float	32 (4)	IEEE single-precision
double	64 (8)	IEEE double-precision
long double	80–128 (10–16)	Extended precision

Fixed-Width Integers (stdint.h)

Type	Size (bits/bytes)	Range
int8_t	8 (1)	[-128,+127]
uint8_t	8 (1)	[0,255]
int16_t	16 (2)	[-32768,+32767]
uint16_t	16 (2)	[0,65535]
int32_t	32 (4)	[-2,147,483,648,+2,147,483,647]
uint32_t	32 (4)	[0,4,294,967,295]
int64_t	64 (8)	[-263,+263-1]
uint64_t	64 (8)	[0,264-1]

Fast / Minimum-Width Types

Type	Size	Description
int_least8_t	$\geq$8	Smallest type $\geq$8 bits
int_fast8_t	$\geq$8	Fastest type $\geq$8 bits
int_least16_t	$\geq$16	Smallest $\geq$16 bits
int_fast16_t	$\geq$16	Fastest $\geq$16 bits
int_least32_t	$\geq$32	Smallest $\geq$32 bits
int_fast32_t	$\geq$32	Fastest $\geq$32 bits
int_least64_t	$\geq$64	Smallest $\geq$64 bits
int_fast64_t	$\geq$64	Fastest $\geq$64 bits

4.4 Memory and Pointers

Memory is a large array of bytes addressed from 0 to $2^{64} - 1$. Variables occupy consecutive bytes.

C Code	Bytes	Meaning
int i;	4	Integer stored in memory
int *p;	8	Pointer to an integer
&i	8	Address of integer i
*p	4	Dereference pointer (value of i)
int **q;	8	Pointer to pointer
*q	8	Pointer to integer
**q	4	Integer value

Dynamic Memory Allocation

C Code	Meaning
float *q;	Pointer declared, value undefined
q = malloc(sizeof(float));	Allocate memory for 1 float
q = malloc(100*sizeof(float));	Allocate memory for 100 floats
free(q);	Free allocated memory

Pointer Arithmetic

C Code	Meaning
*p = 3.14;	Set first float
p[0] = 8.1;	First float = 8.1
p[1] = -4.0;	Second float = -4.0
*(p+1) = -4.0;	Equivalent to previous line
q = p + 10;	q points to 11th float
q[-2] = 555.5;	9th float set

Golden Rule: `p[i]` $\equiv$ `*(p+i)`

4.5 Top Command and Monitoring

- `top`: Shows system load, running tasks, CPU and memory usage
- Example: `top -u munge -H` shows all threads for user `munge`

4.6 Redirection and Pipelines

- `ls *.c > output.dat`: Save output of `ls` to a file
- `cat output.dat`: View file contents
- `program1 | program2 | program3`: Output of one program becomes input to the next

4.7 Other Useful Commands

- `more / less`: View files page by page
- `ls`: List directory contents
- `wc`: Count lines, words, and characters

4.8 Arguments in C

C programs can accept command-line arguments via the `main` function:

```
#include <stdio.h>
int main(int argc, char *argv[]) {
    // argc: number of arguments
    // argv: array of argument strings
}
```


`argc` counts the program name and all arguments, `argv[0]` is the program name, and `argv[1..argc-1]` contain the passed arguments.

4.9 Wildcards (Globbing)

Wildcards allow pattern matching in file names:

Wildcard	Description	Example
*	Matches zero or more characters	*.c (matches test.c, example.c)
?	Matches exactly one character	*.? (matches a.c, b.g)
"[abc]"	Matches one character in the set	*.[ch] (matches file.c or file.h)
"[a-z]"	Matches a range of characters	[a-z]*, [a-zA-Z][0-9]*

4.10 File Permissions

Linux files have permissions for user, group, and others:

- `r`: read, `w`: write, `x`: execute
- Example:

  ```
  -rw-r--r-- 1 dpotter uzh0 0 test_file
  ```

  User can read/write, group can read, others can read.
- Change permissions with `chmod`:

  ```
  chmod 755 test_file   # rwx for user, rx for group & others
  chmod g-rx,o-rx file  # remove group & other permissions
  ```

4.11 Conditional Statements in Bash

`if/then/else` is used to control flow:

```
if <command>; then
    <commands>
elif <command>; then
    <commands>
else
    <commands>
fi
```

Boolean Logic in Bash

- : AND
- `||`: OR

Examples:

```
if true && false; then echo yes; fi   # nothing
if true || false; then echo yes; fi   # prints yes
if true && echo second; then echo yes; fi   # prints second then yes
```

4.12 Functions in Bash

Functions encapsulate reusable code:

```
function distance() {
    local x=$1
    local y=$2
    echo "scale=10;sqrt($x*$x + $y*$y)" | bc
    return 0
}
```

- Use `local` for variables scoped to the function
- Functions can return a status code (integer) and/or print results
- Arguments to functions are accessed as 1,2, ...

4.13 Searching with grep

`grep` searches for patterns in files:

- Basic usage: `grep "pattern" filename`
- Useful flags:
  - `-i`: ignore case
  - `-r`: recursive search
  - `-n`: show line numbers
  - `-v`: invert match

4.14 Bash Regular Expressions

- Regular expressions allow complex pattern matching in scripts
- Can be used with `grep`, `[[ ]]`, or `sed/awk`
- Example: capture numeric values from a string

```
if [[ "$line" =~ ([0-9]+) ]]; then
    echo "Found number: ${BASH_REMATCH[1]}"
fi
```


5 OpenMP and Multi-Threading

5.1 Overview of OpenMP

OpenMP provides **shared-memory parallelism** using threads within a single process.

- A program starts as a single process, called the **master thread**.
- The master thread can create lightweight worker threads at parallel regions.
- Each thread executes a block of instructions, sharing or privately storing variables.
- Variables:
  - **Private**: local to each thread (stack memory)
  - **Shared**: located in main RAM, accessible by all threads

5.2 Parallel Regions and the Fork-Join Model

- Sequential code is executed only by the master thread.
- Parallel regions execute code by all threads.
- **Fork-Join**: Master thread "forks" child threads at parallel regions; they execute tasks and "join" back to the master after completion.
- Creating threads is relatively expensive and introduces latency.

5.3 Work Sharing Strategies

OpenMP allows three main types of work distribution:

1. Loop-level parallelism: divide iterations of a loop among threads
2. Sections: different code blocks executed by different threads
3. Task replication: multiple instances of the same procedure, one per thread

5.4 Directives, Clauses, and Environment Variables

- **Directives**: Special `#pragma omp` lines that guide the compiler to create parallel regions, assign work, and synchronize variables.
- **Clauses**: Modify behavior of directives (e.g., `private`, `firstprivate`, `lastprivate`, `reduction`).
- **Environment variables**: e.g., `OMP_NUM_THREADS` sets the number of threads, `OMP_SCHEDULE` controls loop scheduling.

5.5 Parallel Loops in OpenMP

- A parallel loop distributes independent iterations across threads.
- Loop indices are always private.
- Default behavior includes an implicit barrier at the end of the loop.
- Example:

```
#ifdef _OPENMP
#include <omp.h>
#endif
#include <stdio.h>
int main() {
    int n = 4096;
    #pragma omp parallel
    {
        int rank = omp_get_thread_num();
        #pragma omp for
        for(int i = 0; i < n; ++i) {
            printf("%2d %5d\n", rank, i);
        }
    }
    return 0;
}
```

Scheduling Strategies

- `static, N`: loop divided into fixed-size chunks, assigned round-robin.
- `dynamic, N`: chunks assigned to threads as they become available.
- `guided, N`: exponentially decreasing chunk sizes, larger than N.
- `runtime`: scheduling determined at runtime using `OMP_SCHEDULE`.

5.6 Variable Scoping in Loops

- `private(a)`: variable local to each thread, uninitialized.
- `firstprivate(a)`: thread-local copy initialized with shared value.
- `lastprivate(a)`: thread-local copy, final value updated to shared variable after loop.

5.7 Exclusive Execution Directives

- `sections`: assign consecutive blocks to different threads.
- `single`: execute block with one arbitrary thread; optional `copyprivate` to share results.
- `master`: executed only by master thread, no barrier at end.

5.8 Atomic and Critical Updates

- `atomic`: ensures single-thread access for simple shared variable updates.
- `critical`: protects more complex updates or code sections from race conditions.

5.9 Memory Access and Performance

Memory hierarchy strongly affects OpenMP performance:

1. CPU registers, L1 cache (fastest)
2. L2/L3 cache
3. Main RAM on same socket
4. Main RAM on different socket (NUMA)

- Poor cache management and false sharing can degrade performance.
- On Linux, memory is mapped to a socket on a "first touch" basis; optimizing data placement improves NUMA performance.
- Measure performance using `omp_get_wtime()` for elapsed time in seconds.

6 Message Passing Interface (MPI)

6.1 Overview of MPI

MPI is a library for **distributed-memory parallelism** that allows coordination of thousands to millions of processes:

- Each process has its own private memory.
- Processes can execute different parts of a program independently.
- Communication occurs via message-passing routines.
- MPI contrasts with OpenMP, which is **shared-memory** within a node.

6.2 Sequential vs. Message-Passing Model

- **Sequential programming**: one process, all variables in local memory, executed on a single CPU.
- **Message-passing programming**: multiple processes, private local memory, communication via messages.

6.3 Message Passing Concepts

- Messages are sent from a source to a target process (sender and receiver).
- Message attributes:
  - Sender ID (rank)
  - Receiver ID (rank)
  - Data type
  - Data length
  - Message tag
- Messages are managed by the MPI runtime system (like a postal system).
- Messages must be interpreted correctly by receiving processes.

6.4 Distributed vs. Shared Memory

- MPI: distributed memory, data transferred explicitly over network.
- OpenMP: shared memory, data implicitly available to all threads on the node.

6.5 Communicators, Rank, and Size

- All MPI communications happen within a **communicator**.
- The default communicator is `MPI_COMM_WORLD`, including all active processes.
- Each process has a **rank** within the communicator.
- Total number of processes is called the **size**.

6.6 Point-to-Point Communications

- Occurs between a **sender** and a **receiver** process.
- Both are identified by their rank in the communicator.
- The message header includes:
  - Sender rank
  - Receiver rank
  - Message tag
  - Communicator ID
  - Data type
- Transfer modes correspond to different protocols (blocking, non-blocking, synchronous, buffered).

6.7 MPI Datatypes

Datatype	Description
MPI_CHAR	character
MPI_INT	integer
MPI_FLOAT	single-precision floating point
MPI_DOUBLE	double-precision floating point
MPI_BYTE	raw byte data
MPI_LONG	long integer

6.8 Blocking Point-to-Point Communication

MPI_Send and **MPI_Recv** provide basic blocking communication:

- `MPI_Send(buf, count, datatype, dest, tag, comm)`
- `MPI_Recv(buf, count, datatype, source, tag, comm, status)`
- Execution is blocked until the message is fully sent/received.
- **Wildcards**: `MPI_ANY_SOURCE`, `MPI_ANY_TAG` allow flexible matching.
- **MPI_PROC_NULL** ignores the communication.
- **MPI_STATUS_IGNORE** can be used to skip status info.

Deadlock Warning: Blocking sends/receives can deadlock if a process waits for a message that will never arrive.

6.9 Collective Communications

- Involve all processes in a communicator.
- Examples:
  - `MPI_BARRIER()` – global synchronization
  - `MPI_BCAST(buf, count, datatype, root, comm)` – broadcast
  - `MPI_REDUCE(buf, result, count, datatype, op, root, comm)` – reduction to one process
  - `MPI_ALLREDUCE(buf, result, count, datatype, op, comm)` – reduction to all processes
  - `MPI_SCATTER / MPI_GATHER` – distribute/collect data
  - `MPI_ALLGATHER` – gather to all processes
  - `MPI_ALLTOALL` – all-to-all communication

Reduction Operations

MPI provides predefined operations:

Operation	Description
MPI_SUM	Sum of all elements
MPI_PROD	Product of all elements
MPI_MAX	Maximum value
MPI_MIN	Minimum value
MPI_BAND	Bitwise AND
MPI_BOR	Bitwise OR

6.10 Non-blocking Communication

- `MPI_Isend`, `MPI_Irecv` return immediately.
- Use `MPI_Wait` or `MPI_Test` to ensure completion.
- Allows overlapping computation with communication.
- Avoids deadlocks, but requires careful buffer management.

6.11 Ghost Communication (Halo Exchange)

- Used in domain decomposition (e.g., finite difference grids).
- Each process exchanges boundary data with neighbors.
- Example (Fortran pseudo-code):


```
call MPI_ISEND(boundary_data, ...)  ! send to neighbor
call MPI_IRECV(buffer, ...)         ! receive from neighbor
call MPI_WAIT(...)                   ! wait for completion
```

6.12 Derived Datatypes

- Allows sending complex structures as a single MPI message.
- `MPI_TYPE_CREATE_STRUCT()` maps C/Fortran structs to MPI types.
- `MPI_GET_ADDRESS()` extracts memory offsets.
- Helps portability across architectures and compilers.
- Example: define `rowtype` and `columntype` for 2D arrays:

```
// C example
MPI_Type_vector(ncols, 1, ncols_total, MPI_DOUBLE, &columntype);
MPI_Type_commit(&columntype);
```

6.13 Data Distribution and Synchronization

- Distribute data to maximize local computation and minimize communication.
- Collectives can simplify code and reduce risk of deadlocks.
- Always be aware of synchronization points and barriers.
- Use derived datatypes when sending complex or structured data.

7 Memory Wall and Parallel Scaling

7.1 Von Neumann Architecture and Bottleneck

- Program instructions and data share a single memory unit.
- CPU loads instructions and data from memory, executes them, and stores results back.
- **Von Neumann Bottleneck:**
  - Memory bandwidth grows slower than CPU speed.
  - Memory latency decreases slowly.
  - More CPU cores compete for the same memory resources.
- **Consequences:** CPU cycles are wasted waiting for data.
- **Solutions:**
  - Cache hierarchy: L1, L2, L3 caches.
  - Parallel access to memory: vectorized instructions (e.g., AVX).

7.2 Amdahl's Law (1967)

- Defines theoretical maximum speedup for a parallelized program:

$$\text{Speedup}(N) = \frac{T_{\text{serial}}}{T_{\text{parallel}}} = \frac{1}{\alpha + \frac{1-\alpha}{N}}$$

- T_{serial}: execution time of serial code T_{parallel}: execution time with N processors α: fraction of code that cannot be parallelized
- Maximum speedup is limited by the non-parallel fraction:

$$\text{Speedup}(N) \to \frac{1}{\alpha} \quad \text{as } N \to \infty$$

7.3 Strong vs. Weak Scaling

- **Strong scaling:** Fix total problem size, increase number of cores.
  - Example: 1 billion elements on 100 cores $\to$ 10 million elements per core
- **Weak scaling:** Increase problem size proportionally with number of cores.
  - Example: 100 billion elements on 100 cores $\to$ 1 billion elements per core

7.4 Evolution of Parallel Programming Methods

- **MPI:** Still dominant for distributed memory parallelism.
- **Hybrid OpenMP/MPI:** Effective for modern supercomputers.
- **GPU programming:** CUDA and OpenACC; message passing directly on GPU.
- **New programming models:** Co-array Fortran, PGAS, X10, Chapel.
- **Task-based runtime systems:** Charm++, HPX.

8 Hybrid Computing and GPU Programming

8.1 Memory Types: DRAM vs SRAM

Type	Description	Notes
DRAM	1 capacitor + 1 transistor per bit	Needs refresh; slow; large capacity; inexpensive
SRAM	6 transistors per bit	No refresh; very fast; expensive; used for caches

8.2 Cache Memory and Levels

- **L1 cache:** Smallest, fastest, per core.
- **L2 cache:** Larger, slightly slower, per core or shared among few cores.
- **L3 cache:** Largest, shared among many cores, slower but faster than RAM.
- Goal: Keep frequently used data close to CPU to minimize memory latency.

8.3 Non-Uniform Memory Access (NUMA)

- Memory physically attached to each CPU socket.
- Access to local memory is faster than remote memory.
- Optimizing memory placement improves performance.

8.4 SIMD and AVX

- **SIMD:** Single Instruction, Multiple Data — process multiple data elements with one instruction.
- **AVX:** Advanced Vector Extensions for Intel Xeon (e.g., Gold 6126).
- Keep data aligned, in vector format, and avoid divergence for maximum performance.

8.5 CPU vs GPU Comparison

	CPU	GPU
Purpose	General purpose	Optimized for parallel tasks
Memory	Large main memory	Smaller main memory
Clock speed	High	Lower per thread
Latency	Low (caches)	Hidden with many threads
Compute resources	Few cores, fast	Many cores, high throughput
Performance/Watt	Lower	Higher

8.6 GPU Performance Considerations

- Global memory bandwidth can be a bottleneck (e.g., P100 vs A100: 732 vs 1555 GB/s).
- Each warp of 32 threads executes the same instruction (or waits).
- Shared memory is faster than global memory; use to reduce latency.
- High thread count per SM (Streaming Multiprocessor) needed to hide latency.
- Register sharing limits the number of threads per SM.

8.7 CUDA Programming Basics

- Hardware: P100, compute capability 6.0, 56 SMs, max 1024 threads per block.
- Organize threads in **blocks** and blocks in a **grid**.
- Decide number of threads per block and number of blocks to match hardware capabilities.
- Example: For `cpi` problem, choose 64 threads per block, 500 blocks total.

9 GPU Programming with OpenACC

9.1 Overview of OpenACC

- OpenACC is a directive-based API to offload computations to GPUs.
- Directives are added to C/C++/Fortran code; without compiler flags (`-acc`), they are treated as comments.
- Supports data management, parallelism, and synchronization.
- Functions and environment variables allow fine control of parallel execution and data movement.

9.2 Important GPU Operations

- Allocate/free GPU memory
- Transfer data between host (CPU) and device (GPU)
- Launch GPU kernels
- OpenACC can manage transfers automatically; operations can overlap.

9.3 Basic OpenACC Directives

- `#pragma acc parallel`: creates a parallel region on the GPU.
- `#pragma acc kernels`: lets the compiler generate kernels for GPU execution.
- `#pragma acc loop`: indicates loops that can be executed in parallel.
- Clauses such as `private`, `reduction`, `collapse(n)` control variable scope and parallel execution.

9.4 Data Management

- **Structured data regions:** start and end clearly defined in code.
- **Unstructured data regions:** use `enter data` and `exit data` (e.g., in constructors/destructors).
- Scalars and loop indices are private by default; arrays are shared by default.
- Explicit clauses (`private`, `copy`, etc.) can override defaults.

9.5 Hierarchical Parallelism

- **Gangs:** groups of threads executed on different compute units.
- **Workers:** threads inside a gang.
- **Vectors:** smallest unit of execution inside a worker.
- Use clauses `num_gangs(n)`, `num_workers(n)`, `vector_length(n)` to control sizes.

9.6 Loop Constructs and Collapse

- Loops can be parallelized with `loop` directive; iterations must be independent.
- `collapse(n)` combines nested loops into one for better parallel granularity:

```
#pragma acc parallel loop collapse(2)
for(int i=0; i<n; ++i)
  for(int j=0; j<m; ++j)
      ... // loop body
```

9.7 Reduction Operations

- Avoid race conditions when updating shared variables in parallel loops.
- Example: Parallel PI computation

```
#pragma acc parallel
#pragma acc loop reduction(+:sum) private(x)
for(int i=0; i<steps; i++) {
  x = (i+0.5)*step;
  sum += 4.0 / (1.0 + x*x);
}
```


9.8 Synchronization and Atomic Operations

- Implicit synchronization occurs at the end of `kernels` or `parallel` regions.
- Inside a parallel region, loop execution order is not guaranteed.
- `atomic` ensures a shared variable is updated by only one thread at a time.
- Useful to avoid race conditions; may impact performance.

9.9 Asynchronous Programming

- By default, OpenACC is synchronous: host waits for GPU to finish.
- Using `async` clauses, multiple kernels or data transfers can run concurrently.
- Conceptually similar to CUDA streams (queues), allowing overlapping of host computations, GPU kernels, and data transfers.

9.10 Example: SAXPY

- Compute $Y = aX + Y$, vectors of floats, scalar a.
- Serial, OpenMP, and GPU/OpenACC implementations differ in parallelism and memory handling.
- Optimizations: keep data on GPU, minimize host-device transfers.

9.11 Compilation

- Compiler flags (nVidia/PGI):

```
$ CC -O3 -acc -Minfo=acc -o saxpy saxpy.cxx
```

- Enables OpenACC, shows compiler info, and optimizes for GPU execution.

10 CUDA Programming

10.1 Introduction to CUDA

- CUDA exposes GPU parallelism for general-purpose computing while retaining high performance.
- Based on C/C++ with GPU-specific extensions.
- Provides an API to manage devices, memory, and kernels.

10.2 Processing Flow

1. Copy input data from CPU memory to GPU memory via PCIe.
2. Execute GPU kernels with data cached on-chip for performance.
3. Copy results back to CPU memory.

10.3 Hello World Example

```
// Host-only code
int main(void) {
    printf("Hello World!\n");
    return 0;
}
```

Compile: `nvcc hello.cu -o hello`

10.4 Device Code Example

```
__global__ void mykernel(void) {
    printf("Hello from GPU!\n");
}

int main(void) {
    mykernel<<<1,1>>>(); // launch kernel: 1 block, 1 thread
    cudaDeviceSynchronize(); // ensure GPU finishes before host exits
    return 0;
}
```

Notes:
- `__global__` indicates device function called from host.
- Kernel launch syntax: `<<<numBlocks, numThreadsPerBlock>>>`.

10.5 Memory Management

- Host and Device memory are separate.
- Device pointers (`int *d`) point to GPU memory; cannot be dereferenced on host.
- Host pointers (`int *h`) point to CPU memory; cannot be dereferenced on device.
- CUDA API:
  - `cudaMalloc(d, size)` – allocate device memory.
  - `cudaFree(d)` – free device memory.
  - `cudaMemcpy(d, h, size, cudaMemcpyHostToDevice)` – copy memory.

10.6 Parallel Vector Addition

```
__global__ void add(int *a, int *b, int *c, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n) c[idx] = a[idx] + b[idx];
}

int main() {
```



```
    int *a_d, *b_d, *c_d;
    int N = 1000;
    cudaMalloc(&a_d, N*sizeof(int));
    cudaMalloc(&b_d, N*sizeof(int));
    cudaMalloc(&c_d, N*sizeof(int));
    add<<<(N+BLOCKSIZE-1)/BLOCKSIZE, BLOCKSIZE>>>(a_d, b_d, c_d, N);
}
```

Notes:
- Compute global index: `threadIdx.x + blockIdx.x * blockDim.x`.
- Handles arbitrary vector sizes using bounds check (`if (idx < n)`).

10.7 Threads and Blocks

- A **block** contains multiple threads.
- Threads within a block can share fast **shared memory**.
- Synchronization: $_{syncthreads()}ensuresallthreadsreachthebarrierbeforeproceeding.$
- `Optimal performance requires using both multiple blocks and multiple threads per block.`

10.8 Shared Memory Example

```
__global__ void kernel(float *data) {
    __shared__ float cache[BLOCKSIZE];
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    cache[threadIdx.x] = data[idx];
    __syncthreads();
    // perform computations using shared memory
}
```

Notes:
- Shared memory is allocated per block.
- Threads in different blocks cannot access each other's shared memory.
- Avoid placing $_{syncthreads()}inside non-uniform conditional code.$

10.9 Summary

- **Host vs Device:** separate memory spaces, separate code.
- Use `__global__` for kernels, `cudaMalloc`/`cudaMemcpy` for memory management.
- Threads and blocks allow massive parallelism.
- Shared memory and $_{syncthreads()}enablesynchronizationwithinablock.$
- `Proper indexing and bounds checking are crucial for correctness.`

11 Compiler Suites and Profiling Tools

11.1 Compiler Wrappers

- HPC systems often provide **compiler wrappers** like `cc`, `CC`, `ftn`, `mpicc`.
- A wrapper is a small script around the actual compiler that:
  - Automatically includes necessary include paths and libraries for the system
  - Adds the correct MPI/OpenMP libraries when compiling parallel programs
  - Simplifies compilation commands
- **Example:** Compile a C MPI program

```
mpicc -O3 -o myprogram myprogram.c
```

The `mpicc` wrapper links the system MPI library and adds compiler flags.

11.2 Debugging and Optimization Flags

- `-g` flag: Include debugging information in the executable (for gdb, profiler tools).
- `-O` or `-O3`: Optimize the code for performance.
- `-Og`: Optimize for debugging; balance between performance and debugging capabilities.
- Always consider the effect of optimization when profiling or debugging.

11.3 Profiling with CrayPAT

- Cray Performance Analysis Tool (CrayPAT) measures runtime behavior and identifies bottlenecks.
- Ground-up explanation:
  1. Load the module: `module load perftools-lite` (basic) or `perftools` (advanced)
  2. Compile your code with `-g` for debug info (helps CrayPAT map performance to source lines)
  3. Run the program through the batch queue system (`sbatch` or `qsub`)
  4. CrayPAT collects data using sampling or instrumentation
  5. Analyze results: time spent in each routine, hotspots, memory usage
- Sampling collects data periodically with minimal overhead.
- Instrumentation adds explicit calls in the code to measure specific regions.

11.4 Intel VTUNE and AMD Prof

- Tools for profiling and performance analysis on Intel or AMD CPUs.
- VTUNE: Measures CPU utilization, cache misses, vectorization efficiency, threading behavior.
- Prof: Similar functionality for AMD processors.

11.5 Self Timing in C/C++

- Measure execution time by capturing timestamps before and after a computation.
- High-resolution timers in C++:

```
#include <chrono>
auto start = std::chrono::high_resolution_clock::now();
// code to time
auto end = std::chrono::high_resolution_clock::now();
double duration = std::chrono::duration<double>(end-start).count();
```

- Useful for profiling specific code regions without external tools.

11.6 Summary

- Compiler wrappers simplify linking and building code on HPC systems.
- Use `-g` and `-O`/`-Og` flags carefully depending on debugging/profiling needs.
- CrayPAT provides sampling and instrumentation-based profiling.
- Intel VTUNE and AMD Prof are alternatives for CPU-level profiling.
- Self-timing is a simple way to measure execution time of specific code regions.

12 Cloud Computing and Containers

12.1 Cloud Services Overview

- **Infrastructure as a Service (IaaS)**: Provide virtualized hardware resources (CPU, memory, storage) over the internet.
- **Anything as a Service (XaaS)**: Extends cloud concepts to software, platforms, and other services.
- **Economy of scale**: Building and maintaining physical data centers is expensive; clouds provide centralized, shared resources.
- **Data centers**: Large, cooled, secure locations with thousands of servers.
- Servers are **virtualized**: Multiple virtual machines (VMs) can run on a single physical server.

12.2 Containers as a Service (CaaS)

- Similar principle to VMs, but for **containers**.
- **Containers**:
  - Lightweight, portable environments
  - Include all software dependencies (OS libraries, binaries, etc.)
  - Do not require a full operating system like VMs
  - Simplify deployment and reproducibility

12.3 Why Containers?

- Ensures consistent environment across different systems.
- Useful when software requires:
  - Specific OS (e.g., RedHat, Ubuntu)
  - Specific library versions
  - Complex compilation process
  - Binary-only distributions
- Solutions:
  - Provide a VM image (heavier)
  - Provide a container (lightweight, portable)

12.4 Example Workflow on UZH Cloud

1. **Create an instance** with the desired hardware configuration.
2. **Login** via SSH:

   ```
   ssh ubuntu@172.23.x.x
   ```

3. **Update package catalog**:

   ```
   sudo apt update
   ```

4. **Install required packages**:

   ```
   sudo apt install cowsay
   sudo apt install gcc cmake
   ```

5. **Create a snapshot** for future reuse (optional).
6. **Automation** using BASH or Python scripts is possible.

12.5 Key Points

- Containers are a lightweight alternative to virtual machines.
- Provide reproducible environments with all dependencies.
- Simplify deployment on cloud or HPC systems.
- Combine with cloud services to scale resources on demand.

13 MapReduce and Hadoop

13.1 Hadoop Overview

Hadoop is a framework for distributed storage and parallel processing of large datasets. Key components:

- **Client**: You, submitting jobs.
- **HDFS (Hadoop Distributed File System)**: Stores data across multiple nodes.
- **NameNode**: Stores metadata (directory structure, file locations).
- **YARN (Yet Another Resource Negotiator)**: Schedules resources and jobs.
- **Data Node**: Stores actual data blocks.
- **Node Manager**: Executes MapReduce jobs, paired with Data Nodes.

13.2 Local Storage Analogy

On your laptop:

- Files are stored with POSIX semantics.
- Multiple drives can be combined (appears as a single drive).
- Data security and reliability can be provided with **RAID**.
- File sharing via network: NFS, Samba (SMB/CIFS), AFP.

13.3 RAID (Redundant Array of Independent/Inexpensive Disks)

- Combines multiple disks into one logical unit for performance and/or reliability.
- **RAID 0 (striping)**:
  - Splits data across multiple drives.
  - Maximizes storage and read/write performance.
  - No redundancy – a disk failure leads to total data loss.
- **RAID 1 (mirroring)**:
  - Duplicates data across drives.
  - Provides fault tolerance – one disk can fail without data loss.
  - Storage capacity is halved.
- **RAID 5 (distributed parity)**:
  - Combines performance, reliability, and cost.
  - Data and parity blocks are spread across multiple disks.
  - Tolerates a single disk failure while keeping most of the storage usable.

13.4 Parallel File System – Lustre

- Provides POSIX-like file semantics.
- Files are distributed across multiple servers (metadata + data separation).
- High single-file and aggregate performance.
- Example speeds:
  - Normal HD: 0.2 GB/s
  - `$SCRATCH` on Lustre: 138 GB/s

13.5 Object Storage – Ceph

- Stores data as objects (blobs), not traditional files.
- Objects are duplicated (typically 3 replicas) across multiple servers.
- Metadata stored separately on SSD.
- Provides resilience without relying on a single server.

13.6 MapReduce Programming Model

- Operates on key-value pairs.
- **Mapper**: Processes input data, emits key-value pairs.
- **Partitioner**: Divides keys into partitions for reducers.

- **Shuffle**: Moves partitions to reducers.
- **Reducer**: Aggregates data by key.
- **Combiner** (optional): Local aggregation before shuffle.

13.7 Hadoop Streaming

- Hadoop is Java-based.
- Mapper and Reducer can be written in any language that reads from `stdin` and writes to `stdout`.
- In this course, we use Python for streaming jobs.

13.8 Example: SSH Port Forwarding for Hadoop Web Interfaces

```
ssh -L 9870:localhost:9870 \
    -L 8088:localhost:8088 \
    ubuntu@172.23.8.109
```

- `-L local_port:remote_host:remote_port` maps local ports to remote Hadoop web UIs.
- Access NameNode (9870) and YARN ResourceManager (8088) from your local machine.

13.9 WordCount Example (Python)

- Mapper reads lines, emits `(word, 1)` pairs.
- Reducer sums counts for each word.
- Example result: a combined count of all words across the dataset.

13.10 Modern Alternatives to Hadoop

- **Apache Spark**: Optimized for batch and interactive queries.
- **Presto/Trino**: SQL queries on distributed data.
- **Dask/Ray**: Python-friendly parallel computing.
- **Apache Flink**: Structured streaming for real-time data.

14 Beyond Von Neumann Architecture and Future HPC

14.1 MPI Evolution

- MPI-1: Core point-to-point and collective communication functions.
- MPI-2:
  - Optimized parallel I/O functions.
  - Single-sided communication for remote memory access (still message-based, not shared memory).
- MPI-3: Introduced **non-blocking collectives** for overlapping computation and communication.
- MPI remains a reliable tool for **coarse-grained parallelism** across distributed memory machines.

14.2 OpenMP Evolution

- Parallel regions and parallel "for" loops form the basics.
- Version 3: Introduced **task** construct for multitasking.
- Version 4: Expanded support for:
  - Heterogeneous systems (CPU + GPU or other accelerators).
  - Thread affinity (binding threads to cores).
  - Vector operations (SIMD/AVX).
- OpenMP enables **fine-grained parallelism** on shared-memory systems.

14.3 Hybrid MPI+X

- Combines coarse-grained MPI with finer-grained OpenMP (or other "X" models).
- Common approach in supercomputing: **MPI+OpenMP**.
- "MPI+X" is a general term for MPI combined with any other local parallel programming model (OpenMP, CUDA, OpenACC, etc.).

14.4 Limits of Moore's Law

- Single-core performance has plateaued.
- Performance growth now comes from:
  - More cores per node.
  - Vectorization (SIMD) on each core.
  - Increasing number of nodes in clusters.

14.5 Road to Exascale Computing

- Trend toward **lightweight, power-efficient nodes** instead of few powerful nodes.
- Heterogeneous architectures: CPU + multiple GPUs per node.
- Examples: TaihuLight nodes, Xeon Phi (discontinued), ARM-based Mont Blanc/Alps.
- Special-purpose accelerators: FPGAs.

14.6 Exascale Challenges

- **Energy efficiency**: 50 Gflops/Watt needed (Frontier supercomputer goal). Energy costs CHF 1M/MW/year.
- **Hardware parallelism**: Need billion-way parallelism (e.g., 100M cores with 10-way SIMD per core).
- **Software**: Support multi-level, billion-way parallelism.
- **Latency**: Longer interconnects in larger systems.
- **Reliability**: Mean Time Between Failures (MTBF) becomes critical.

14.7 Asynchronous Multitasking

- Current models (MPI/OpenMP) often use static work scheduling.
- Asynchronous multitasking:
  - Work is decoupled into independent tasks.
  - Dynamic scheduling assigns tasks to available resources (CPU, GPU, memory).
  - Reduces idle times and improves load balancing.

14.8 Multithreaded Applications and Overdecomposition

- OpenMP provides multithreading: one thread per compute core is common.
- Overdecomposition:
  - Launching more threads than cores (e.g., 1000 threads for 10 cores).
  - Threads operate independently and are dynamically scheduled onto cores.
- Used in GPU programming (CUDA, OpenACC) to hide latency.
- Thread switching overhead must be considered.